PostgreSQL HA Architecture Patterns

Table of Contents

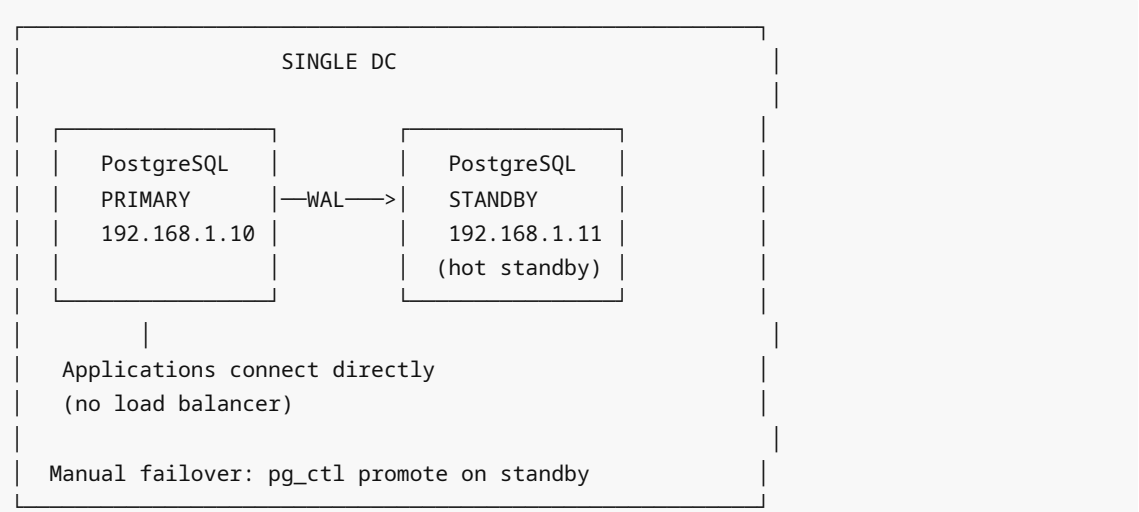
Learning Objectives

By the end of this module, you will be able to:

- Design a complete HA architecture for different scale requirements
- Choose the right pattern based on RTO, RPO, and read scaling needs
- Draw and explain multi-region replication topologies
- Describe each component's role in a full HA stack
- Plan capacity for each layer of the HA stack
- Explain trade-offs between simplicity and resilience

Pattern 1: Single Primary + Hot Standby

Use Case: Small applications, development, budget-constrained **RTO:** 2-5 minutes (manual failover) **RPO:** Seconds to minutes (async replication lag)



Configuration Summary:

```
# Primary postgresql.conf
wal_level = replica
max_wal_senders = 3
max_replication_slots = 3
wal_keep_size = 1GB
```

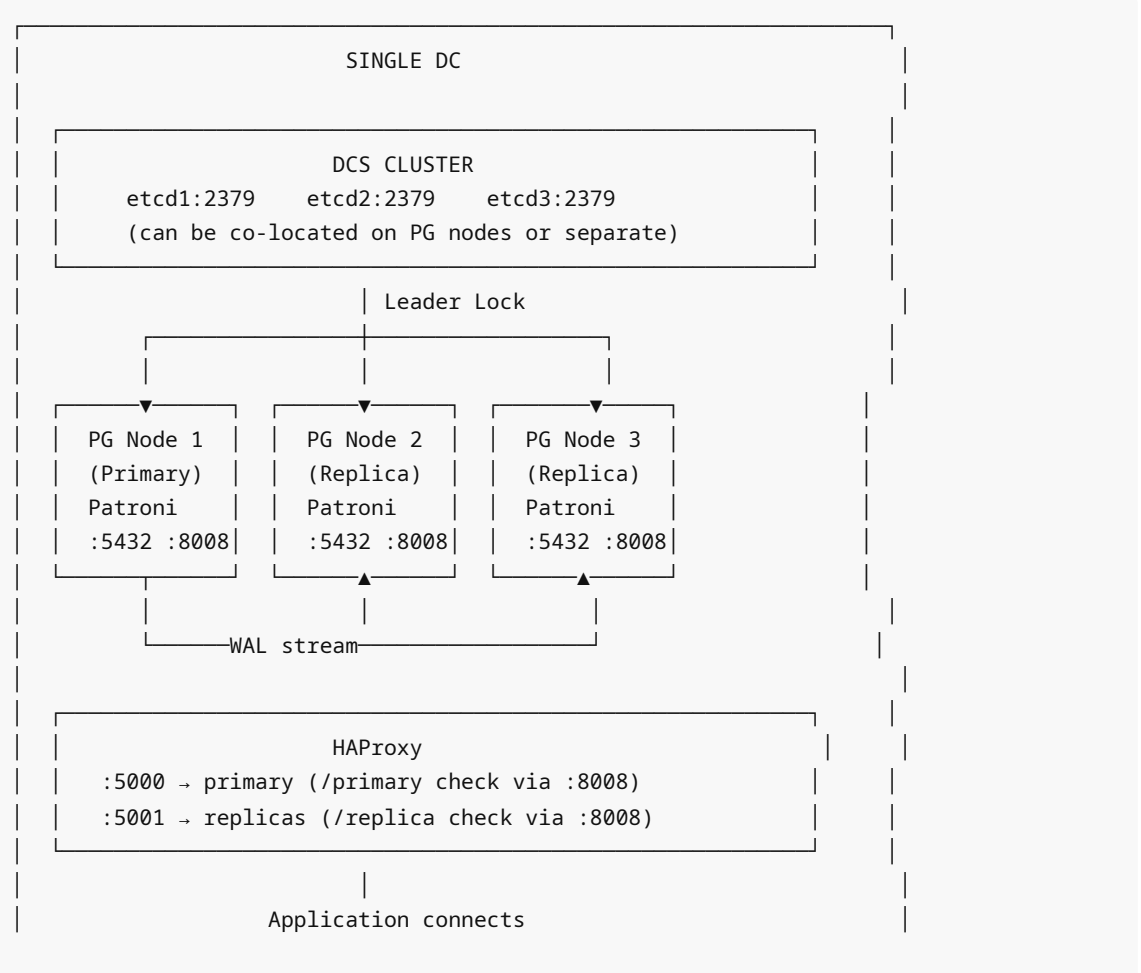
```
# Standby: pg_basebackup + standby.signal + postgresql.auto.conf
primary_conninfo = 'host=192.168.1.10 port=5432 user=replicator'
```

Limitations:

- Manual failover (operator must act)
- Application must update connection string on failover
- Single point of failure until failover is complete

Pattern 2: 1 Primary + 2 Replicas + Patroni

Use Case: Medium applications, need automatic failover **RTO:** 30-60 seconds (automatic failover) **RPO:** 0 with sync replication / seconds with async



```
│                   to HAProxy                                  │
└───────────────────────────────────────────────────────────────┘
```

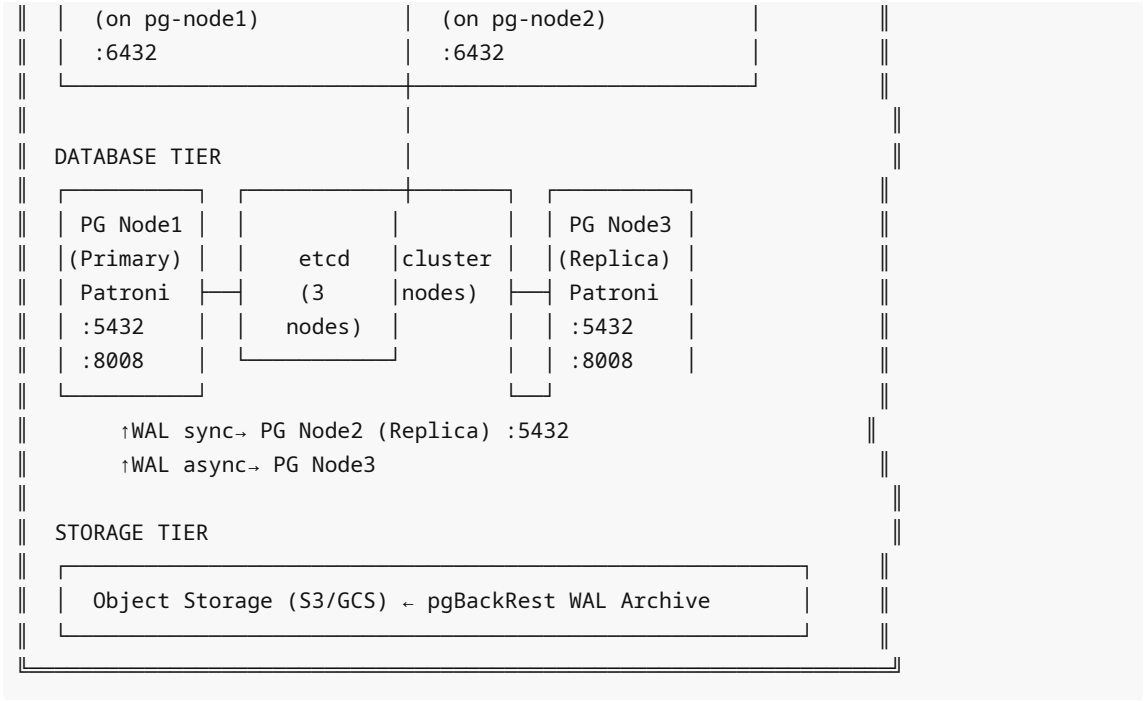
Patroni Configuration:

```yaml
# patroni.yml (per node)
scope: production-cluster
bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    maximum_lag_on_failover: 1048576
    postgresql:
      use_pg_rewind: true
      parameters:
        wal_level: replica
        max_wal_senders: 10
        hot_standby: 'on'
```

Pattern 3: Full HA Stack (Production Standard)

Use Case: Production applications, high availability requirement **RTO:** 15-30 seconds **RPO:** 0 (synchronous) or seconds (async)

```
╔═══════════════════════════════════════════════════════════════════╗
║                   FULL PRODUCTION HA STACK                        ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║  APPLICATION TIER                                                 ║
║  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐        ║
║  │ App Srv1 │  │ App Srv2 │  │ App Srv3 │  │ App Srv4 │        ║
║  └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘        ║
║       └─────────────┴─────────────┴──────────────┘                ║
║                           │                                       ║
║                   VIP: 10.0.0.1                                  ║
║                           │                                       ║
║  PROXY TIER               │                                       ║
║  ┌────────────────┐  ┌────▼───────────────┐                      ║
║  │  HAProxy 2     │  │    HAProxy 1       │                      ║
║  │  (Standby)     │  │    (Active)        │                      ║
║  │  Keepalived    │  │    Keepalived      │                      ║
║  │  prio=100      │  │    prio=200        │                      ║
║  └────────────────┘  └───────────────────┘                       ║
║                           │                                       ║
║                  Port 5000 (writes)                              ║
║                  Port 5001 (reads)                               ║
║                           │                                       ║
║  CONNECTION POOL TIER     │                                       ║
║  ┌────────────────────────┼─────────────────────────┐            ║
║  │  PgBouncer1            │  PgBouncer2              │            ║
```

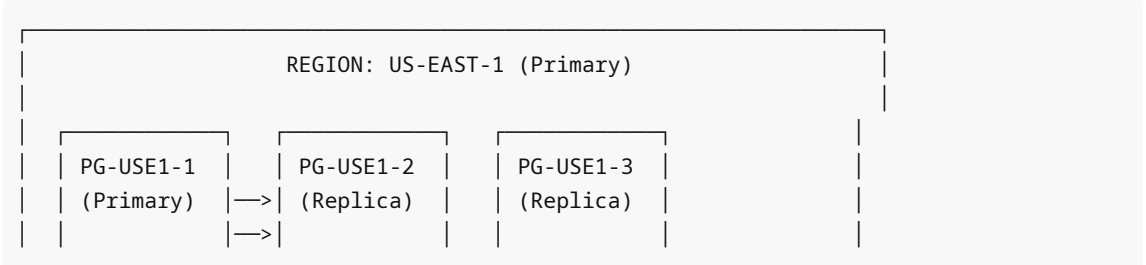


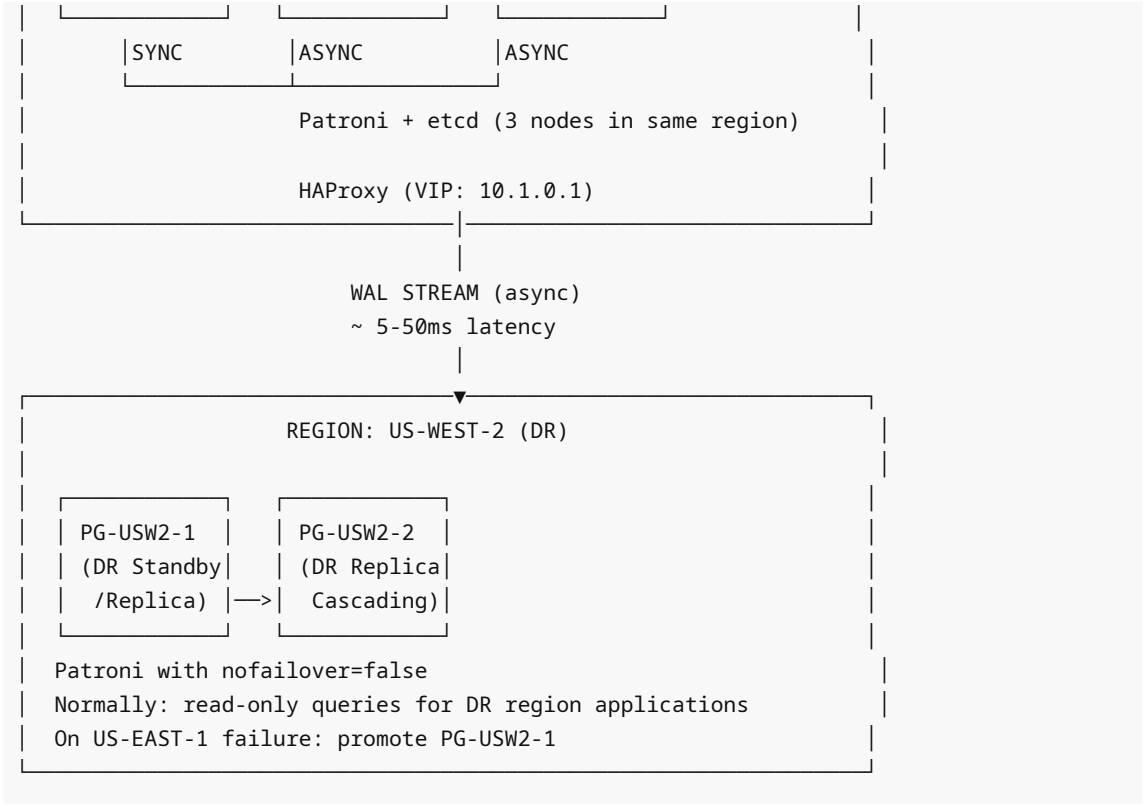
Component Responsibilities:

Layer	Component	Purpose
Application	App servers	Business logic
Network	VIP + Keepalived	Single IP, HAProxy HA
Routing	HAProxy	Route writes/reads, health checks
Pooling	PgBouncer	Multiplex connections
HA Management	Patroni	Leader election, automatic failover
DCS	etcd (3 nodes)	Leader lock, cluster config
Database	PostgreSQL (3 nodes)	Data storage, replication
Backup	pgBackRest + S3	Point-in-time recovery

Pattern 4: Multi-Region Architecture

Use Case: Disaster recovery, geographic distribution **RTO:** Primary region: 30-60s (Patroni) / Cross-region: 5-15 min (manual) or automated **RPO:** Intra-region: 0 / Cross-region: seconds to minutes (async lag)





Cross-Region Replication Config:

```
# DR standby in US-WEST-2 (postgresql.auto.conf)
primary_conninfo = 'host=use1-primary.internal port=5432 user=replicator
application_name=dr_standby'
recovery_min_apply_delay = 0          # Apply immediately (or set delay for lag
protection)
```

```
# Patroni on DR node: prevent from becoming primary automatically
tags:
  nofailover: true        # Do NOT automatically promote this DR node
  noloadbalance: true     # Don't include in read LB pool from primary region
  clonefrom: false
```

DR Failover Procedure:

```
# When US-EAST-1 is down:

# 1. Verify primary region is truly down
# 2. Check DR node lag
psql -h dr-primary -c "SELECT now() - pg_last_xact_replay_timestamp() AS lag;"

# 3. Accept data loss or wait
# If lag > 0: there's potential data loss; decide with stakeholders

# 4. Remove nofailover tag and promote
```

```
patronictl edit-config dr-cluster --set 'tags.nofailover=false'
patronictl failover dr-cluster --master pg-usw2-1 --force

# 5. Update DNS / routing to DR region
# 6. Document and conduct post-mortem
```

Pattern 5: Read-Heavy Architecture

Use Case: Analytics, reporting, read-heavy OLTP **Goal:** Scale reads horizontally

```
                        WRITE PATH
App ──WRITES──> HAProxy :5000 ──> Primary (1 node)

                        READ PATH
App ──READS───> HAProxy :5001 ──> Read Replica 1 (round-robin)
                              ──> Read Replica 2
                              ──> Read Replica 3
                              ──> Read Replica 4

                 ANALYTICS PATH (no SLA)
Analytics ──> HAProxy :5002 ──> Analytics Replica
                              (logical replication, can have indexes)
```

HAProxy Weights for Read Distribution:

```
backend postgres_reads
    balance roundrobin
    option httpchk GET /replica
    http-check expect status 200

    # Equal weight replicas
    server replica1 10.0.1.11:5432 check port 8008 inter 2000 fall 3 rise 2 weight 1
    server replica2 10.0.1.12:5432 check port 8008 inter 2000 fall 3 rise 2 weight 1
    server replica3 10.0.1.13:5432 check port 8008 inter 2000 fall 3 rise 2 weight 1
    server replica4 10.0.1.14:5432 check port 8008 inter 2000 fall 3 rise 2 weight 1
    # Primary as backup for reads (only used if ALL replicas fail)
    server primary  10.0.1.10:5432 check port 8008 backup
```

Cascading for Read Replicas:

```
Primary ──WAL──> Replica1 (sync, HA)
   │
   └──WAL──> Relay1 ──WAL──> ReadReplica1
                          ──> ReadReplica2
                          ──> ReadReplica3
```

Pattern 6: Zero-Downtime Migration Architecture

Use Case: Major PostgreSQL version upgrade, schema migration

```
MIGRATION PHASE:
┌──────────────────┐          Logical Replication          ┌──────────────────┐
│  PG 14 (Old)     │ ──────────────────────────────────────> │  PG 17 (New)     │
│  Production      │                                        │  Staging/New     │
│  PRIMARY         │  Publication: pub_all                  │  SUBSCRIBER      │
│                  │  FOR ALL TABLES                        │  (read-write)    │
│  wal_level=      │                                        │  Schema applied  │
│  logical         │                                        │  manually first  │
└──────────────────┘                                        └──────────────────┘
         │                                                          │
     App writes here                                        Catching up...
     (current production)                                   Monitoring lag

CUTOVER PHASE (5-10 min window):
1. Drain app connections to old primary
2. Verify lag = 0 bytes
3. Disable subscription on new
4. Migrate sequences
5. Switch app to new primary
6. Validate
7. Remove old system
```

Capacity Planning

Formula for Pool Sizing

```
PostgreSQL max_connections budget:
  - Reserve 20% for superuser, monitoring, admin
  - Divide remainder by number of (db, user) pools in PgBouncer

Example:
  max_connections = 500
  Superuser reserve: 500 * 0.2 = 100 connections
  Available for apps: 400 connections
  PgBouncer pools (10 services × 2 envs): 20 pools
  default_pool_size = 400 / 20 = 20 per pool

PgBouncer max_client_conn:
  = number of app workers × 2 (headroom)
  = 500 workers × 2 = 1000 max_client_conn
```

Storage Planning for HA

```
Per-node storage:
  - Data files: X GB
  - WAL: X * 0.05 to 0.10 (WAL is typically 5-10% of data)
  - Replication buffer: wal_keep_size (1-2 GB)
  - Logs: 10-50 GB
  - OS + PostgreSQL binaries: 20 GB
  - Total per node: X * 1.15 + 80 GB headroom
```

```
S3/backup storage:
  - Full backup: X GB
  - Incremental: X * 0.01 per day (1% change per day typical)
  - WAL archive: based on WAL generation rate * retention days
  - Total: X + (X * 0.01 * 30) = 1.3X for 30-day retention
```

Network Topology Considerations

```
LATENCY REQUIREMENTS:
Sync standby: < 5ms RTT (same datacenter or AZ)
Async standby: < 50ms RTT (adjacent region)
DR standby: any (accept RPO = latency)

BANDWIDTH REQUIREMENTS:
WAL generation rate × number of standbys = minimum bandwidth
Example: 100 MB/s WAL × 3 standbys = 300 MB/s required
With wal_compression: × 0.4 = 120 MB/s effective

NETWORK ISOLATION:
Replication traffic → dedicated VLAN (not shared with app traffic)
DCS traffic (etcd) → low-latency, high-reliability network
Backup traffic (pgBackRest) → separate NIC / offpeak scheduling
```

Common Mistakes

1. **Deploying HA without testing failover** — the most common and costly mistake
2. **etcd on same nodes as PostgreSQL** — DCS failure coincides with PG failure
3. **Sync standby too far away** — adds 20-100ms to every transaction commit
4. **No connection pooling** — 1000+ connections overwhelm PostgreSQL
5. **HAProxy as single node** — the HA proxy is itself not HA
6. **Not monitoring replication lag** — discovering hours of lag during failover
7. **Forgetting backup** — HA protects against hardware failure, not logical corruption
8. **Over-engineering small applications** — 3-node Patroni cluster for 10 users
9. **No DR plan** — HA within one DC doesn't protect against DC loss
10. **Not documenting the architecture** — new team members can't operate it

---

Best Practices

1. **Match architecture to requirements** — don't over-build or under-build
2. **Always pair HA with backup** — they solve different problems
3. **Test failover quarterly** — scheduled drills in production-like environments
4. **Monitor all layers** — database, Patroni, PgBouncer, HAProxy, DCS, OS
5. **Use separate DCS** — don't co-locate etcd with PostgreSQL in small clusters
6. **Document runbooks** — ops team must act quickly during incidents
7. **Automate failover** (Patroni) but retain manual override capability
8. **Keep DR standby in read-only mode** for DR region queries, not just a cold standby
9. **Implement connection retry logic** in applications for transparent failover
10. **Use Patroni** for production — manual streaming replication is error-prone at 3am

---

Interview Questions

Q1: Design a PostgreSQL HA architecture for an e-commerce platform handling 10,000 TPS.

A: Full HA stack: (1) 3 PostgreSQL nodes with Patroni (1 primary + 2 replicas), one sync standby for RPO=0 on orders/payments. (2) etcd 3-node cluster on separate servers. (3) 2 HAProxy nodes with Keepalived VIP — port 5000 for writes, port 5001 for reads. (4) PgBouncer per PostgreSQL node (transaction mode, pool_size=25). (5) pgBackRest with S3 for continuous WAL archiving. (6) DR standby in second DC (async). (7) Monitoring: Prometheus + pg_exporter + alerting.

Q2: What is the difference between HA and DR?

A: HA (High Availability) protects against individual component failures (server crash, disk failure) within a single data center. Failover happens in seconds to minutes automatically. DR (Disaster Recovery) protects against entire data center loss. Recovery requires minutes to hours and typically involves manual steps. Both are needed for a complete resilience strategy.

Q3: Why use both HAProxy and PgBouncer?

A: They solve different problems. HAProxy performs intelligent TCP routing: it knows which PostgreSQL node is the primary (via Patroni health checks) and routes writes/reads accordingly. After failover, HAProxy automatically redirects to the new primary. PgBouncer multiplexes application connections: 500 app workers share 20 real PostgreSQL connections, reducing memory pressure and connection overhead. Together they provide routing intelligence AND connection efficiency.

Q4: How does automatic failover work in a Patroni cluster?

A: (1) Primary Patroni fails to renew its leader lock in etcd (either primary or DCS is unreachable). (2) TTL expires (default 30s). (3) Patroni on primary self-demotes by stopping PostgreSQL (preventing split-brain). (4) Replicas detect the expired lock and race to acquire it via atomic CAS in etcd. (5) Winner promotes its PostgreSQL. (6) Loser re-configures as replica of new primary. (7) HAProxy detects new primary within `fall * inter` seconds and routes traffic there.

Q5: What RTO and RPO can you realistically achieve with Patroni?

A: RTO with Patroni: 30-60 seconds (TTL 30s + failover + HAProxy detection ~15s). With synchronous replication: RPO = 0 (no data loss). With async: RPO = replication lag at time of failure (typically seconds). For financial applications requiring RPO=0: use `ANY 2 (s1, s2, s3)` quorum synchronous replication.

Q6: When would you add a read replica versus scaling the primary?

A: Add read replicas when: the workload is read-heavy (>70% reads), read queries are analytically complex (don't benefit from write throughput tuning), or you need geographic distribution of read traffic. Scale the primary when: the bottleneck is write throughput, or queries are tightly coupled (frequent write-read of same rows). Adding replicas doesn't help write performance.

Q7: How does a multi-region architecture handle the split-brain risk during network partition?

A: In multi-region setups with async replication, a network partition between regions risks split-brain if both regions try to elect a primary. Mitigation: (1) Use different Patroni scopes (different DCS clusters) per region, requiring manual intervention to activate DR region. (2) Implement fencing via cloud API to stop the primary region before activating DR. (3) Use a global DCS (across regions) with quorum requiring cross-region consensus — but this adds latency to all commits.

Q8: What is the role of etcd in Patroni HA?

A: etcd provides: (1) Leader lock — only the node holding the lock in etcd can be primary. (2) Cluster configuration — all nodes read shared config from etcd. (3) Member registration — nodes announce themselves. (4) Split-brain prevention — the primary self-demotes if it can't reach etcd, rather than waiting to be fenced externally. etcd must itself be highly available (3+ nodes), as it becomes the availability bottleneck.

Exercises and Solutions

Exercise 1: Architecture Review

Given this architecture, identify the single points of failure:

```
App ──> HAProxy (1 node) ──> PgBouncer (1 node) ──> PG Primary
                                                  ──> PG Replica
```

Solution:

- HAProxy: single node is SPOF → add second HAProxy + Keepalived
- PgBouncer: single node is SPOF → deploy per-PG-node or 2+ PgBouncer instances
- No DCS shown → if using Patroni, need 3-node etcd cluster
- Only 1 replica → need 2+ for quorum-based sync OR reliable async

Exercise 2: Design for RPO=0 with Good Availability

```
Requirement: RPO=0, can tolerate 1 standby failure, max 60s RTO

Solution:
- 1 primary + 3 standbys
- synchronous_standby_names = 'ANY 2 (s1, s2, s3)'
- Patroni with DCS quorum
- HAProxy health checks /primary endpoint

Analysis:
- ANY 2 of 3: even if 1 standby fails, 2 remain for quorum → RPO=0 maintained
- If all 3 standbys fail: primary blocks writes (acceptable if rare)
- Patroni handles failover: ~30-60s RTO
```

Exercise 3: Runbook — Add Read Replica to Existing Patroni Cluster

```bash
#!/bin/bash
# add_read_replica.sh

NEW_NODE_IP=$1
PATRONI_PRIMARY=$(patronictl list | grep Leader | awk '{print $4}')

echo "Current primary: $PATRONI_PRIMARY"
echo "Adding replica at: $NEW_NODE_IP"

# 1. Install PostgreSQL and Patroni on new node
ssh root@$NEW_NODE_IP "apt-get install -y postgresql-16 patroni"

# 2. Copy patroni.yml (update node name and connect_address)
```

```
scp /etc/patroni/patroni.yml root@$NEW_NODE_IP:/etc/patroni/patroni.yml
ssh root@$NEW_NODE_IP "
    sed -i 's/name: .*/name: pg-$(echo $NEW_NODE_IP | tr . -)/'
/etc/patroni/patroni.yml
    sed -i 's/connect_address: .*/connect_address: $NEW_NODE_IP:5432/'
/etc/patroni/patroni.yml
"

# 3. Start Patroni (will automatically pg_basebackup from primary)
ssh root@$NEW_NODE_IP "systemctl start patroni"

# 4. Monitor until it joins as replica
sleep 30
patronictl list

# 5. Update HAProxy to include new replica
# Add server line to /etc/haproxy/haproxy.cfg
# Reload haproxy

echo "New replica added!"
```

Cross-References

- 02_streaming_replication.md — Streaming replication details
- 04_synchronous_replication.md — Sync replication config
- 05_failover_procedures.md — Manual failover steps
- 06_patroni.md — Patroni cluster management
- 07_pgbouncer.md — Connection pooling
- 08_haproxy_load_balancing.md — Load balancer config
- ../15_Backup_Recovery/06_pgbackrest.md — Backup integration